

Lab 4: VibeCoding - Chat History

Student: Shreya Yashodhara Singh **SRN:** PES1UG24AM266 **GitHub:** yasho26shreya

Assigned repo: SETAPESU26/27_centipede (Centipede Repair Lab, Pygame) **LLM used:** Claude (Claude Code session)

Chat link: https://claude.ai/code/session_01R2t4hjtyL3todBxQ2TdbvD

This is the record of the single chat session used for the lab. The student's message is verbatim; Claude's replies and tool work are condensed into the steps below. The chat link above has the complete original history.

Turn 1 - Student prompt (verbatim)

[attached: Lab_4_VibeCoding_Student_handout.pdf, VibeCodingGitRepoLinks_E.xlsx]
my name is shreya yashodhara singh..and my srn ia pes1ug24am266..do all the deliverables and my github is yasho26shreya

Claude - work log

- 1. Read the handout and the repo list.** The handout asks for: before/after 10-second videos, updated code with a separate commit per task, and the chat history as a doc/pdf, all under a Lab-4 folder; no PR to the main repo. In the spreadsheet, the number AM266 maps to row 27, 27_centipede. (The sheet lists the SRN as PES1UG23AM266 while the student gave PES1UG24AM266, so this was flagged for confirmation.)
- 2. Cloned the repo and read README + game.py.** README: one deliberate bug (mushroom durability) and three stub functions: `mushroom_color(hp)`, `on_segment_hit(segment, score)`, `wave_speed_bonus(wave)`.
- 3. Recorded the BEFORE video** on the unmodified game using a scripted player bot on a virtual display (the bot only plays; game.py is untouched). The video shows a mushroom that is removed after 3 shots instead of 4 (before.mp4).
- 4. Task 1 - bug fix.** In `hit_mushroom`, `if self.mushrooms[cell] <= 1` removed a 4-HP mushroom on the 3rd hit. Changed to `<= 0`. Commit 1.
- 5. Task 2 - mushroom_color.** Added `flash_mushroom(hp)` (called from `hit_mushroom`) and made `mushroom_color(hp)` return white for 120 ms after a hit, otherwise None (default fading colour). Because the function only receives HP, the flash matches on HP, so two mushrooms with the same HP can flash together. Unit-tested headlessly. Commit 2.
- 6. Task 3 - on_segment_hit.** Sparks (10 green for a body shot, 20 golden for a head shot) and a floating +10 / +100 label at the hit segment's position. `split_chain` sets `segment.was_head`; effects are updated in `Game.update`, drawn in `Game.draw` and cleared on reset. Tested headlessly (head/body score, chain splits into two non-empty chains, effects expire). Commit 3.
- 7. Task 4 - wave_speed_bonus.** Returns `1 + 0.15 * (wave - 1)`. Measured: the centipede made 33 moves in 3 s on wave 1 and 43 on wave 3 (ratio 1.30, as expected). Commit 4.
- 8. Recorded the AFTER video.** Shows the mushroom surviving 3 shots and clearing on the 4th, white hit flashes, sparks and +10 labels, and wave 2 starting at x1.15 speed (after.mp4). Frames were checked by eye.

Commits (one per task)

Hash	Message
71fba6f	Task 1: fix mushroom durability (survive MUSHROOM_HP hits, clear at hp <= 0)
a1b00fe	Task 2: implement mushroom_color - mushrooms flash white when hit

f77ce72	Task 3: implement on_segment_hit - sparks and +points popup on every segment hit
d8f0d75	Task 4: implement wave_speed_bonus - each wave is 15% faster

Appendix - full code diff (starting point 90853d2 to final)

```
diff --git a/game.py b/game.py
index b6b6e0e..8b4988d 100644
--- a/game.py
+++ b/game.py
@@ -8,19 +8,84 @@ TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
     MUSHROOM_HP = 4

+FLASH_MS = 120
+_flash = {"hp": None, "until": 0}
+
+def flash_mushroom(hp):
+    """Remember that a mushroom was just hit and now has `hp` left, so it can flash white briefly."""
+    _flash["hp"] = hp
+    _flash["until"] = pygame.time.get_ticks() + FLASH_MS
+
+def mushroom_color(hp):
+    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""
+    pass
+    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default.
+
+    A freshly hit mushroom flashes white for FLASH_MS milliseconds. The function only receives the
+    remaining HP, so the flash is matched on HP: any mushroom with exactly that HP flashes.
+    """
+    if hp == _flash["hp"] and pygame.time.get_ticks() < _flash["until"]:
+        return (255, 255, 255)
+    return None
+
+sparks = [] # short-lived particles: [x, y, vx, vy, life, colour]
+popups = [] # floating score labels: [x, y, life, text, colour]
+SPARK_LIFE = 0.5
+_popup_font = None

def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus points here."""
    pass
    """Called whenever a centipede segment is shot: burst of sparks plus a floating +points label.

    A head shot (flagged by split_chain as segment.was_head) gets a bigger, golden burst.
    """
    x = segment.col * CELL + CELL // 2
    y = segment.row * CELL + CELL // 2
    head = getattr(segment, "was_head", False)
    colours = [(255, 230, 90), (255, 255, 255), (255, 160, 40)] if head else [(120, 255, 130), (255, 255, 255), (80, 220, 90)]
    for _ in range(20 if head else 10):
        angle = random.uniform(0, 6.2832)
        speed = random.uniform(60, 220 if head else 160)
        direction = pygame.math.Vector2(1, 0).rotate_rad(angle)
        vx, vy = direction.x * speed, direction.y * speed
        sparks.append([x, y, vx, vy, SPARK_LIFE * random.uniform(0.6, 1.0), random.choice(colours)])
    popups.append([x, y - 6, 0.8, "+100" if head else "+10", colours[0]])

def update_sparks(dt):
    """Advance and expire sparks and popups (called every frame from Game.update)."""
    for sp in sparks:
        sp[0] += sp[2] * dt
        sp[1] += sp[3] * dt
        sp[4] -= dt
    sparks[:] = [sp for sp in sparks if sp[4] > 0]
    for pu in popups:
        pu[1] -= 30 * dt
        pu[2] -= dt
    popups[:] = [pu for pu in popups if pu[2] > 0]

def draw_sparks(screen):
    """Draw sparks and popups (called from Game.draw)."""
    global _popup_font
    for x, y, _vx, _vy, life, colour in sparks:
        size = 2 if life < SPARK_LIFE * 0.4 else 3
        pygame.draw.rect(screen, colour, (int(x), int(y), size, size))
    if popups and _popup_font is None:
        _popup_font = pygame.font.Font(None, 22)
    for x, y, _life, text, colour in popups:
        label = _popup_font.render(text, True, colour)
        screen.blit(label, label.get_rect(center=(int(x), int(y))))

def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default speed."""
    pass
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default speed.

    Wave 1 runs at normal speed (x1.00); every later wave is 15% faster than the base speed
    """
```

```

+ (wave 2 = x1.15, wave 3 = x1.30, ...).
+ """
+ return 1 + 0.15 * (wave - 1)

class Segment:
@@ -47,6 +112,8 @@ class Game:

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
+        sparks.clear()
+        popups.clear()
        self.mushrooms = {}
        for _ in range(45):
            self.mushrooms[(random.randint(1, ZONE_TOP - 2), random.randint(0, COLS - 1))] = MUSHROOM_HP
@@ -70,9 +137,11 @@ class Game:

    def hit_mushroom(self, cell):
        self.mushrooms[cell] -= 1
-        if self.mushrooms[cell] <= 1:
+        if self.mushrooms[cell] <= 0:
            del self.mushrooms[cell]
            self.score += 5
+        else:
+            flash_mushroom(self.mushrooms[cell])

    def split_chain(self, chain, index):
        segment = chain[index]
@@ -81,6 +150,7 @@ class Game:
        self.chains.extend(part for part in (left, right) if part)
        self.mushrooms[(segment.row, segment.col)] = MUSHROOM_HP
        self.score += 100 if index == 0 else 10
+        segment.was_head = index == 0
        on_segment_hit(segment, self.score)

    def update_bullet(self, dt):
@@ -102,6 +172,7 @@ class Game:
        return

    def update(self, dt, keys):
+        update_sparks(dt)
        if self.state != "play":
            return
        self.invulnerable = max(0.0, self.invulnerable - dt)
@@ -140,6 +211,7 @@ class Game:
        for index, segment in enumerate(chain):
            center = (segment.col * CELL + CELL // 2, segment.row * CELL + CELL // 2)
            pygame.draw.circle(screen, (240, 200, 60) if index == 0 else (80, 220, 90), center, CELL // 2)
+        draw_sparks(screen)
        if self.bullet:
            pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1, self.bullet.y - 6, 3, 10))
        if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 == 0:

```